LEMBAR SOAL PRAKTIKUM
MODUL 07: AJAX DAN KONSUMSI API

Nama: Steven Taufik Fajar NIM: 103122400068

Kelas: SE-08-08 Tanggal: 11/5/2026

Kerjakan dua soal proyek berikut. Kedua program secara bersama-sama mencakup seluruh materi Modul 07. Gunakan data contoh dan jalankan melalui browser.

Cakupan dan bobot

Proyek	Materi yang diterapkan	Bobot
Soal 1	Konsep dan alur AJAX; event handling; XMLHttpRequest; Fetch; API lokal/eksternal; loading dan error.	50 poin
Soal 2	Axios GET dan POST; loading/error; useState dan useEffect pada React.	50 poin

Persiapan dan cara menjalankan

1. Gunakan Paket_Praktikum_AJAX.zip dari modul sebelumnya. Ekstrak seluruh paket dan pastikan Node.js 18 atau lebih baru tersedia.
2. Di dalam paket, buat latihan-praktikan/soal-01 dan latihan-praktikan/soal-02. Simpan kode masing-masing proyek pada folder tersebut.
3. Jalankan JALANKAN.bat atau node server.cjs dari folder paket. Biarkan terminal berjalan.
4. Buka http://127.0.0.1:3007/latihan-praktikan/soal-01/ atau alamat soal-02/. Sesuaikan port jika server memakai port lain.

API dan data yang digunakan

Metode dan URL	Keterangan
GET /api/users	Array tiga pengguna: id, name, email.
GET /api/users?mode=empty	HTTP 200 dengan array kosong.
GET /api/users?mode=error	HTTP 503 untuk simulasi gagal.
POST /api/users	Kirim JSON name dan email; HTTP 201, id 101, simulated true. Data tidak disimpan.
GET https://jsonplaceholder.typicode.com/users	API eksternal; membutuhkan internet.

Tambahkan parameter delay dalam milidetik, maksimal 5000. Contoh: /api/users?delay=1500 atau /api/users?mode=error&delay=1000. Mock lokal dapat dipakai tanpa internet.

Ketentuan umum

Simpan data yang ditampilkan berdasarkan response. Tampilkan pesan yang mudah dipahami, gunakan label input, dan tangani kegagalan. Referensi kode boleh memakai contoh pada modul. Skenario uji dan ketentuan pengumpulan tersedia pada halaman 4.

Soal 01. Portal informasi praktikum

Materi: AJAX, event, XMLHttpRequest, Fetch, API lokal/eksternal | Bobot: 50 poin | Folder: soal-01

Laboratorium membutuhkan portal untuk membaca pengumuman dan daftar peserta tanpa memuat ulang seluruh halaman. Buat satu halaman HTML dan JavaScript dengan dua bagian yang saling melengkapi: pengumuman dan direktori peserta.

Berkas yang dibuat

Buat index.html, app.js, dan pengumuman.txt di folder soal-01. Isi pengumuman.txt sebagai berikut:

```
PENGUMUMAN PRAKTIKUM
Praktikum dimulai pukul 08.00.
Bawa kartu praktikum dan rapikan komputer setelah selesai.
```

Kebutuhan program

1. Sediakan kolom Catatan praktikan, tombol Tambah hitungan, angka awal 0, dan waktu halaman pertama dibuka. Catatan, hitungan, dan waktu awal harus tetap ada setelah request selesai.
2. Pada bagian pengumuman, sediakan tombol Muat pengumuman. Event klik harus meminta pengumuman.txt menggunakan XMLHttpRequest secara asynchronous. Periksa readyState dan status HTTP, lalu tampilkan responseText pada elemen pre.
3. Pada bagian direktori, sediakan pilihan sumber Mock lokal atau JSONPlaceholder dan tombol Ambil peserta. Gunakan Fetch untuk mengambil data dari URL sesuai pilihan.
4. Untuk mock, gunakan /api/users?delay=1500. Untuk sumber eksternal, gunakan URL JSONPlaceholder pada halaman 1. Periksa response.ok sebelum membaca JSON.
5. Tampilkan maksimal tiga pengguna pertama beserta emailnya, nama sumber, dan jumlah data yang ditampilkan. Gunakan data dari response; jangan mengisi nama peserta secara manual.
6. Saat menunggu, tampilkan pesan loading dan nonaktifkan tombol request yang sedang berjalan. Tombol Tambah hitungan tetap harus dapat digunakan. Aktifkan kembali tombol request setelah proses selesai.
7. Tampilkan pesan gagal untuk file tidak ditemukan, HTTP gagal, atau gangguan jaringan. Jika API eksternal gagal, arahkan pengguna untuk memeriksa internet atau memilih mock secara manual.

Hasil akhir yang diharapkan

Pengumuman dan direktori dapat diperbarui tanpa reload. Praktikan masih dapat mengetik catatan serta menambah hitungan ketika data sedang diminta. Hasil pengumuman berasal dari XHR, sedangkan daftar peserta berasal dari Fetch.

Penjelasan singkat dalam laporan

Jelaskan event pemicu, request yang dikirim, response yang diterima, dan elemen DOM yang berubah. Tunjukkan bukti bahwa interaksi hitungan tetap berjalan selama menunggu data.

Soal 02. Aplikasi registrasi peserta React

Materi: Axios GET/POST, loading/error, useState, useEffect | Bobot: 50 poin | Folder: soal-02

Buat satu aplikasi React untuk menampilkan daftar peserta dan mengirim formulir pendaftaran. Daftar dimuat otomatis, sedangkan data pendaftaran dikirim setelah pengguna menekan tombol.

Berkas dan library

Buat index.html dan app.jsx di folder soal-02. Siapkan elemen root pada body. Muat React, ReactDOM, Axios, dan Babel berurutan dari ../../vendor/, memakai nama file yang sama dengan contoh React pada modul. Muat app.jsx dengan tag berikut; pemasangan ReactDOM.createRoot() dilakukan di app.jsx.

```
<script type="text/babel" data-presets="react" src="app.jsx"></script>
```

Kebutuhan program

1. Gunakan useState untuk data peserta, loading, error, input formulir, dan status pengiriman. Gunakan useEffect untuk GET awal dan saat pilihan skenario berubah.
2. Ambil daftar memakai axios.get() dan baca response.data. Tampilkan nama serta email menggunakan key dari user.id. Sediakan tombol Muat ulang agar request dapat dijalankan lagi.
3. Sediakan pilihan Berhasil, Data kosong, Error HTTP, dan Timeout. Gunakan URL serta batas tunggu pada tabel di bawah. Tampilkan loading, hasil, kosong, atau pesan gagal sesuai kondisi sebenarnya.
4. Rapikan effect saat pilihan berubah atau komponen dilepas, misalnya dengan AbortController dan penanda effect aktif. Response lama tidak boleh menimpa hasil pilihan terbaru.
5. Sediakan form Nama, Email, dan tombol Kirim pendaftaran. Cegah reload dengan preventDefault(), tolak nama kosong atau hanya spasi, dan validasi format email.
6. Kirim objek name dan email memakai axios.post("/api/users", objekData). Nonaktifkan tombol saat mengirim. Tampilkan status HTTP, id, nama, email, simulated, serta pesan Simulasi berhasil, data tidak disimpan.
7. Tangani kegagalan GET dan POST, lalu pulihkan kontrol setelah selesai. Pendaftaran tidak menambah data ke server; tampilkan response POST terpisah dari daftar GET.

Skenario GET	URL	Timeout Axios
Berhasil	/api/users?delay=1000	5000 ms
Data kosong	/api/users?mode=empty&delay=1000	5000 ms
Error HTTP	/api/users?mode=error&delay=1000	5000 ms
Timeout	/api/users?delay=2500	700 ms

Gunakan try, catch, dan finally untuk mengelola kondisi proses. Batas waktu 700 ms harus menghasilkan kegagalan timeout sebelum response yang dijeda 2500 ms tiba.

Pengujian, pengumpulan, dan penilaian

Jalankan skenario berikut

Soal	Pengujian	Hasil yang harus terlihat
1	Muat pengumuman, lalu ubah isi file dan muat lagi.	Isi baru tampil melalui XHR.
1	Ubah sementara URL pengumuman menjadi hilang.txt.	Pesan gagal dengan status 404. Kembalikan URL setelah uji.
1	Ambil mock sambil mengetik dan klik hitungan tiga kali.	Tiga peserta tampil; catatan tetap ada, hitungan bertambah, waktu awal tetap sama.
1	Pilih API eksternal dan kirim request.	Maksimal tiga pengguna tampil atau pesan gagal yang jelas. Sumber tidak berganti otomatis.
2	Buka halaman, lalu klik Muat ulang.	GET berjalan otomatis saat awal; klik memicu request baru.
2	Coba Data kosong dan Error HTTP.	Pesan kosong atau HTTP 503; daftar lama dibersihkan.
2	Coba Timeout, lalu kembali ke Berhasil.	Request timeout pada batas 700 ms; aplikasi dapat digunakan lagi.
2	Ganti pilihan ketika request masih berjalan.	Hasil akhir mengikuti pilihan terbaru.
2	Kirim Rina Praktikan, rina@example.com.	HTTP 201; id 101; data sesuai input; simulated true. GET tetap tiga peserta.
2	Nama hanya spasi atau format email tidak valid.	Form menolak pengiriman dan memberi petunjuk perbaikan.

Jika internet atau layanan eksternal tidak tersedia, catat kondisi tersebut. Penanganan kegagalan menjadi bukti uji jalur eksternal. API lokal dan POST di atas tetap dapat diuji tanpa internet.

Berkas yang dikumpulkan

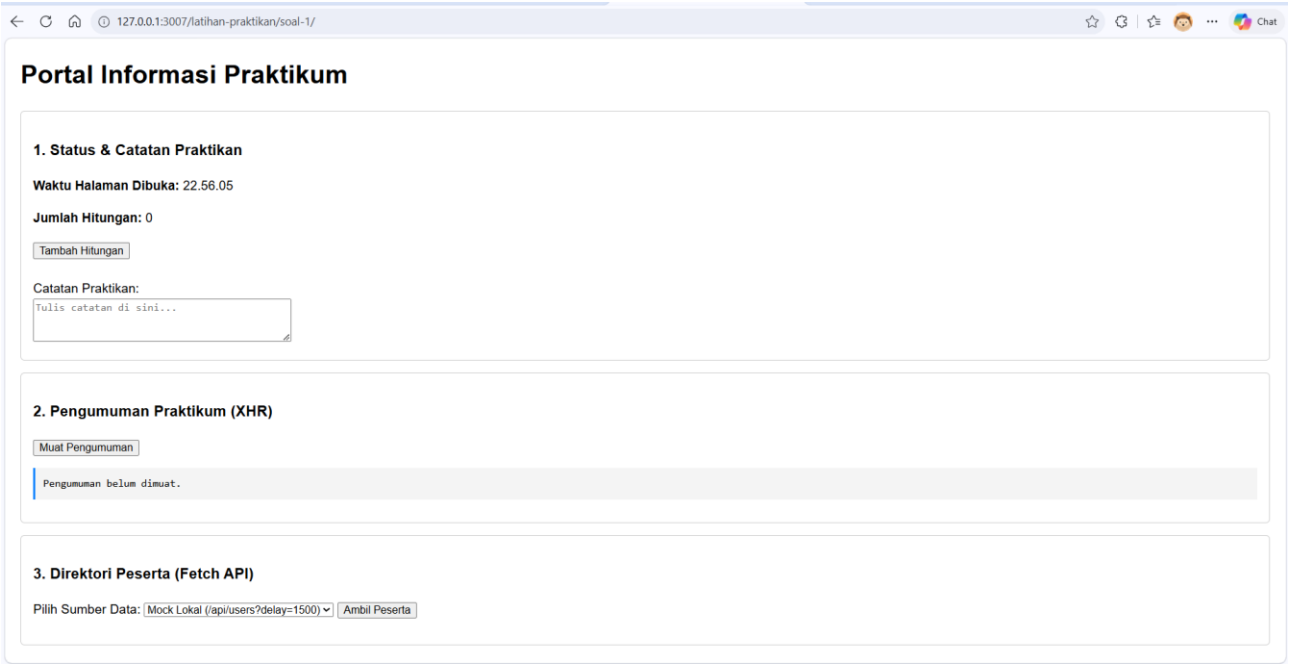
Kumpulkan NIM_Nama_Modul07.zip berisi folder soal-01, soal-02, README cara menjalankan pada paket awal, dan laporan hasil uji. Sertakan tangkapan layar kedua program, bukti POST, serta satu kondisi gagal. Catat hasil aktual setiap skenario dan jelaskan alur XHR, Fetch, Axios, serta peran state/effect React secara singkat.

Rubrik penilaian (nilai akhir maksimal 100 poin)

Aspek	Soal 1	Soal 2
Fungsi utama sesuai kebutuhan	30	30
Loading, validasi, dan penanganan kondisi	10	10
Keterbacaan kode dan bukti pengujian	10	10
Jumlah	50	50

JAWAB:

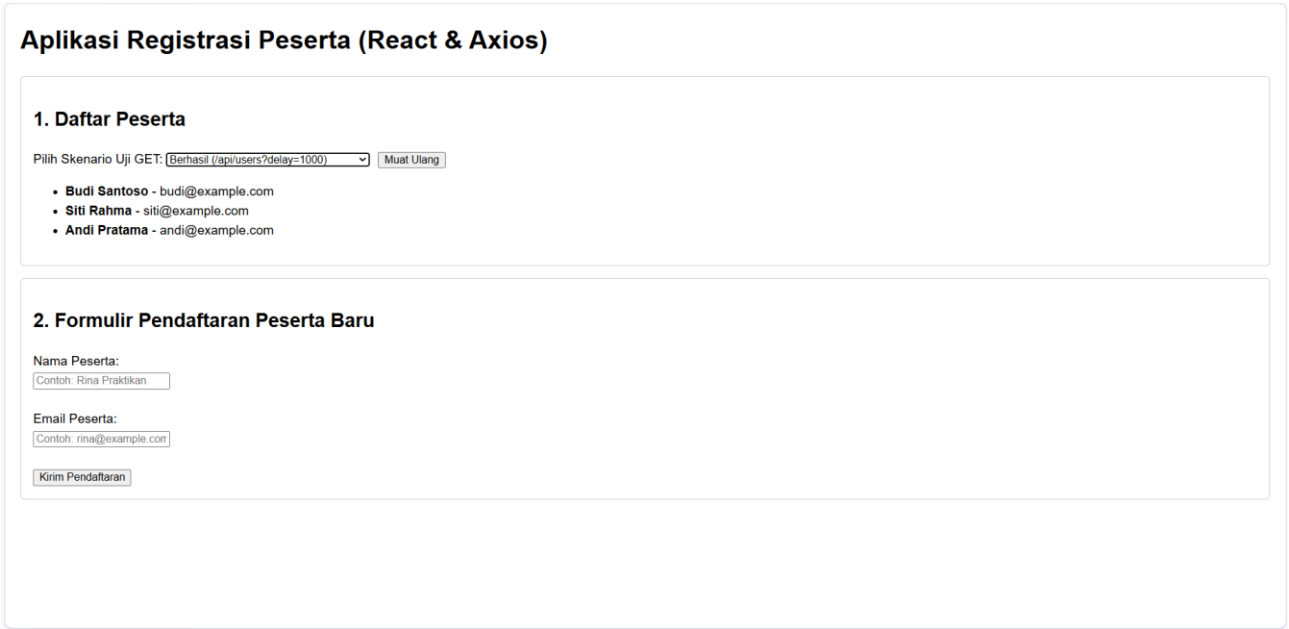
1. ALUR KERJA SOAL 01 (XHR & FETCH API)



XMLHttpRequest (XHR):
Tombol "Muat Pengumuman" membuat event click yang menjalankan instance XMLHttpRequest request dikirim secara asynchronous via method GET ke file "pengumuman.txt" Listener `onreadystatechange` memantau proses hingga readyState === 4 (DONE) dan status HTTP === 200 (OK) lalu menampilkan responseText pada elemen <pre>.

Fetch API & Asynchronous:
Tombol "Ambil Peserta" memanggil fungsi async dengan Fetch API ke endpoint /api/users?delay=1500. Sebelum membedah data JSON via response.json() program memeriksa response.ok. Data dibatasi maksimal 3 peserta Sifat asynchronous terbukti karena pengguna tetap dapat mengetik pada kolom catatan dan menambah hitungan tombol tanpa mengalami freeze/freeze UI saat request sedang diproses.

2. ALUR KERJA SOAL 02 (REACT & AXIOS)

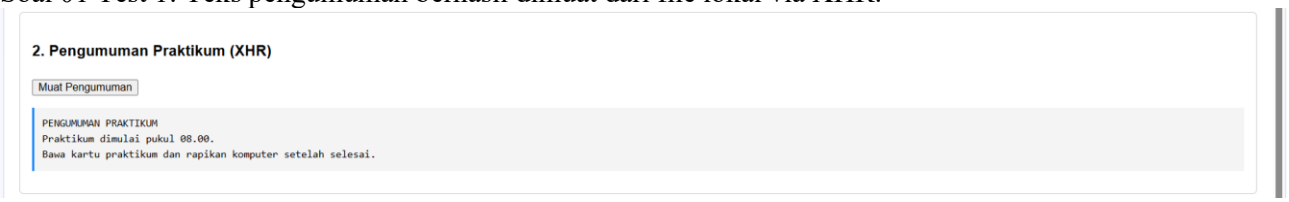


Axios GET & Penanganan Skenario:
Axios otomatis melakukan parse respons JSON dari server pada skenario timeout konfigurasi timeout diset 700 ms untuk memotong request delay 2500 ms, yang ditangkap oleh blok catch() untuk menampilkan pesan error timeout. AbortController digunakan di dalam useEffect untuk membersihkan (cleanup) request lama saat skenario berubah secara cepat.

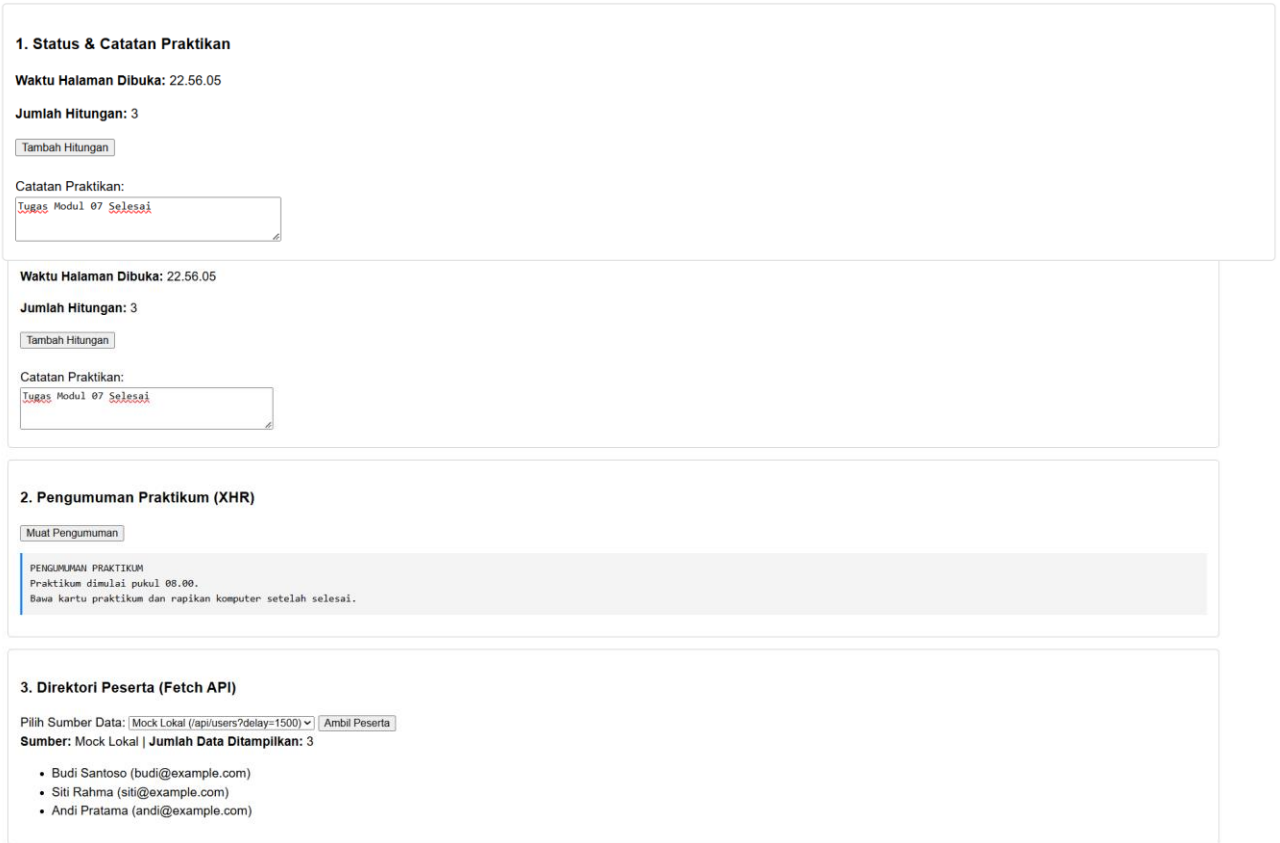
Axios POST Formulir:
Event submit pada form ditangani dengan e.preventDefault() untuk mencegah reload halaman setelah lulus validasi input nama dan email, data dikirim via axios.post("/api/users" objekData). Server mengembalikan respons HTTP status 201 dengan ID simulasi 101 yang ditampilkan terpisah di UI.

Peran useState dan useEffect:
useState: Menyimpan data dinamis aplikasi (daftar peserta, status loading, pesan error, serta nilai input form) yang otomatis memperbarui tampilan UI React saat berubah.
useEffect: Mengeksekusi fungsi GET secara otomatis saat halaman pertama kali dibuka dan melakukan request ulang setiap kali state dependency (skenario) berganti.

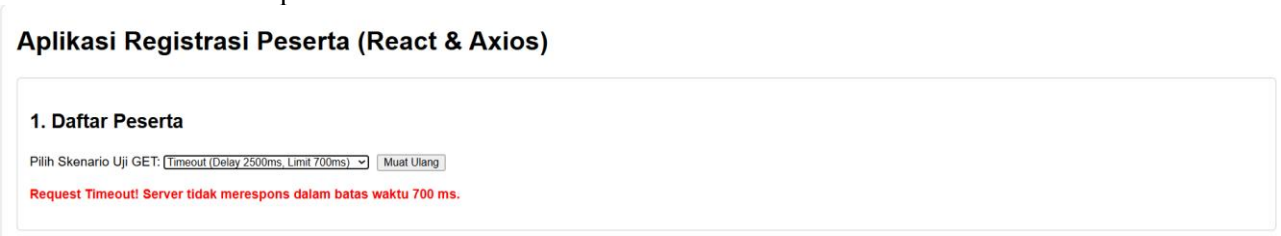
HASIL PENGUJIAN
Soal 01 Test 1: Teks pengumuman berhasil dimuat dari file lokal via XHR.



Soal 01 Test 2 & 3: Tiga peserta dari Mock Lokal tampil; angka hitungan (3) dan isi catatan tetap utuh selama request berlangsung.



Soal 02 Test 1: Skenario Timeout memicu pesan error "Request Timeout! Server tidak merespons dalam batas waktu 700 ms." tepat setelah 700 ms.



Soal 02 Test 2: Formulir pendaftaran berhasil mengirim payload POST dan menampilkan respons HTTP status 201 dengan ID 101.

